

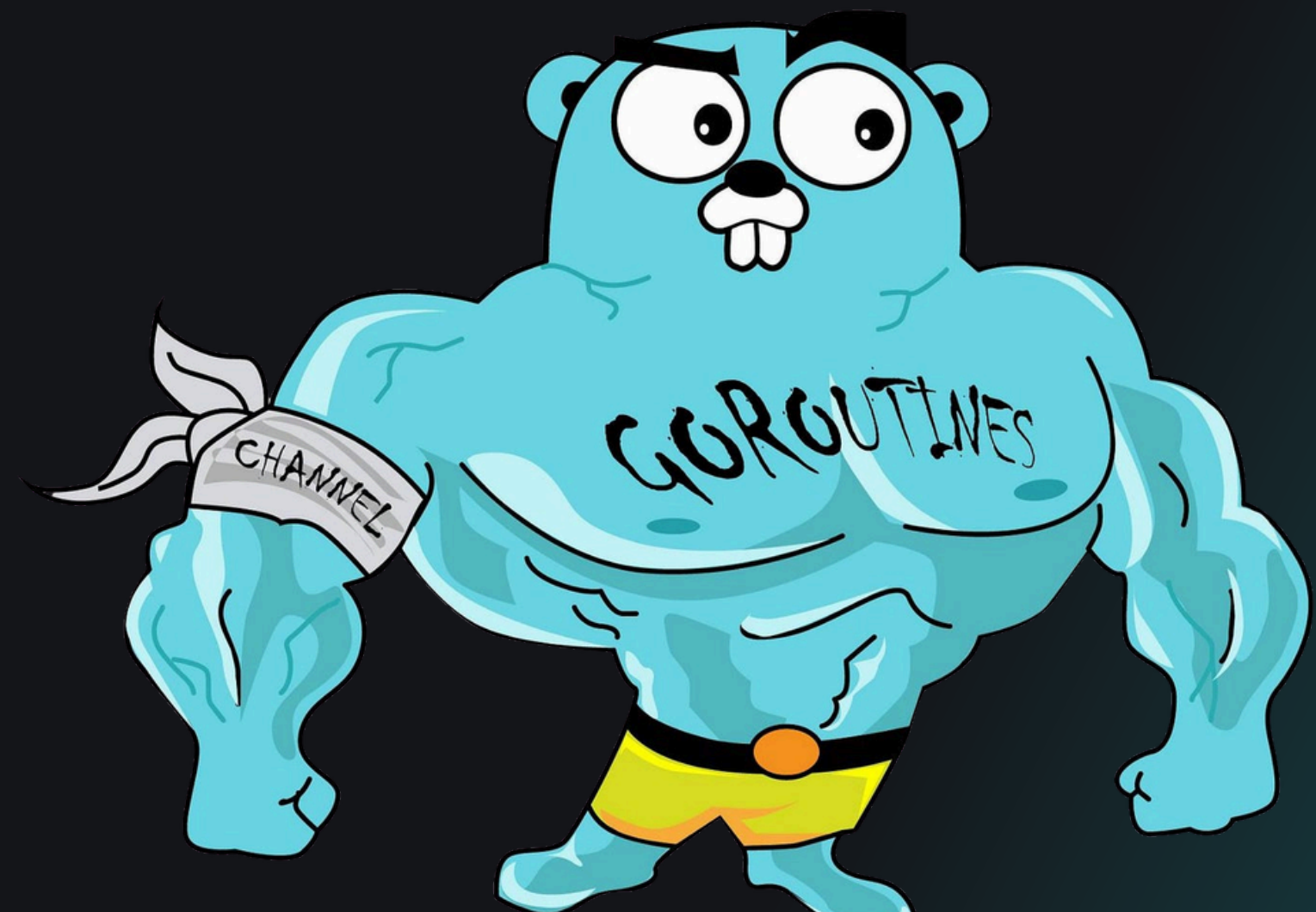
Conceptos para adapters

Por: Ricardo Cuellar



Paquete sync

El paquete sync contiene herramientas para coordinar trabajo concurrente entre goroutines



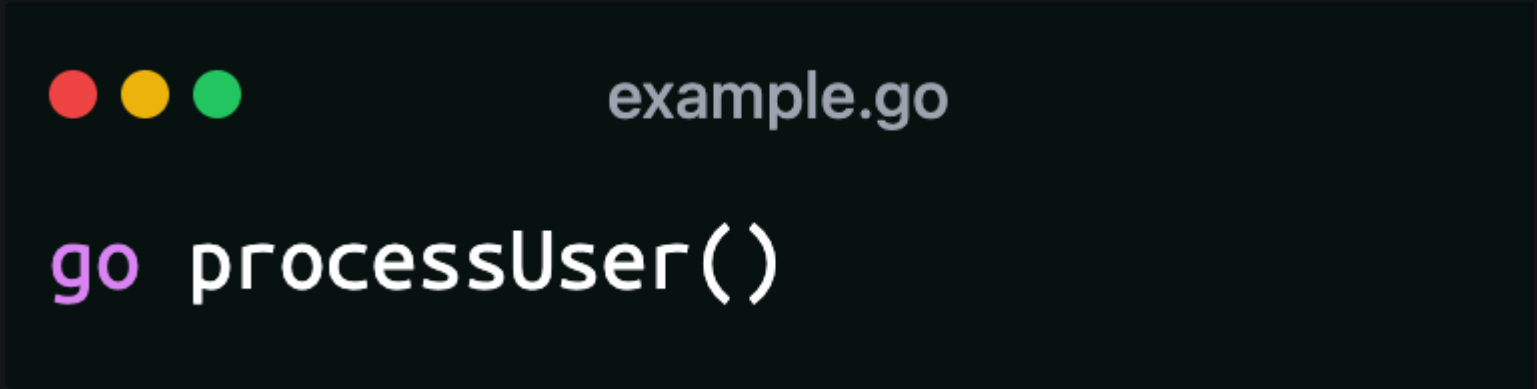
Herramientas en sync

- sync.Mutex
- sync.RWMutex
- sync.Once
- sync.WaitGroup
- sync.map



¿Qué es una goroutine?

Es una función que Go puede ejecutar concurrentemente con otras

A dark-themed terminal window with three colored window control buttons (red, yellow, green) in the top-left corner. The text "example.go" is in the top-right corner. The code "go processUser()" is written in the terminal, with "go" in purple and "processUser()" in white.

```
example.go  
  
go processUser()
```

Nota: en un servidor HTTP en Go no se necesita escribir "go" ya los procesa de forma concurrente.



goroutine en Go Server

- Request 1 → Crea usuario Ana
- Request 2 → Busca usuario Ricardo
- Request 3 → Busca email ana@email.com
- Request 4 → Crea usuario Fulanito

Si todo se llama hacia un mismo lugar (map) aquí aparece el problema de concurrencia. Tenemos que proteger a map.



¿Y por qué debo proteger a map?

Porque los maps no son seguros para lecturas y escrituras concurrentes.

Goroutine A → `r.users["user_1"] = ana`

Goroutine B → lee `r.users["user_1"]`

fatal error: concurrent map read and map write



¿Qué es un Mutex?

Mutex significa *mutual exclusión*, es decir, exclusión mutua.



Mutex



Goroutine A

Goroutine B

`mu.Lock()` → Bloquea

....

`mu.Unlock()` → Libera



r.mu.Lock()

Lock() significa: desde este punto, ninguna otra goroutine puede obtener un bloqueo de lectura o escritura sobre este RWMutex hasta que lo libere.

r → Repositorio

mu → Mutex dentro del
repositorio

Lock() → Bloqueo de escritura



r.mu.Unlock()

Unlock() significa: Liberar el bloqueo adquirido mediante Lock()

Si no lo usamos las goroutines quedarían esperando indefinidamente que podría producir un bloqueo llamado **deadlock**

mu.Lock() → Bloquea

.... trabajo ...

mu.Unlock() → Libera



Buena práctica para Unlock()

Usamos defer para hacer que Unlock() se ejecute después de que termine la función, sin importar si el camino es bueno o algo falló.

```
example.go  
  
r.mu.Lock()  
defer r.mu.Unlock()
```



sync.RWMutex

Significa: Read-Write Mutex y distingue dos tipos de acceso: lectura y escritura.

Con Mutex:

Lectura A → Entra
Lectura B → espera
Lectura C → espera

Con RWMutex:

Lectura → RLock()
Escritura → Lock()



Reglas de `sync.RWMutex`

- Varias lecturas pueden ocurrir al mismo tiempo
- Solo una escritura puede ocurrir a la vez
- Mientras hay escritura, no puede haber lecturas
- Mientras hay lectores activos, la escritura debe esperar.



Mutex vs RWMutex

Mutex: todo el acceso es exclusivo, aunque sea solo lectura.



example.go

```
mu.Lock()
```

```
defer mu.Unlock()
```



Mutex vs RWMutex

RWMutex: para lecturas y escrituras



example.go

```
mu.RLock()  
defer mu.RUnlock()
```

Lectura



example.go

```
mu.Lock()  
defer mu.Unlock()
```

Escritura



RW/Mutex

RLock + RLock + RLock



Lock + Lock



Lock + RLock



Gracias



www.devtalles.com



[/ricardocuellars](https://www.linkedin.com/company/ricardocuellars)



[@ricardocuellars](https://twitter.com/ricardocuellars)

